Введение

В настоящее время методы машинного обучения и глубокого обучения активно применяются для анализа клиентских данных в сфере маркетинга и управления взаимоотношениями с клиентами. Такие методы позволяют выявлять скрытые зависимости между поведенческими характеристиками клиента и его потенциальной ценностью для бизнеса. Одной из актуальных задач является прогнозирование пожизненной ценности клиента (Customer Lifetime Value, CLV) на основе истории его покупок, поведенческих и социально-демографических данных.

CLV является ключевым показателем, позволяющим оценить долгосрочную прибыльность клиента и оптимизировать маркетинговые стратегии. Поэтому программные системы прогнозирования CLV могут использоваться как вспомогательный инструмент для принятия решений о распределении маркетинговых бюджетов, персонализации предложений и удержании клиентов. При этом такие системы не заменяют экспертные решения маркетологов и не предназначены для окончательной оценки клиента, а могут применяться только для информационной оценки вероятной будущей ценности клиента.

Целью данной контрольной работы является разработка программного приложения в виде веб-сервиса с HTML-интерфейсом для прогнозирования ценности клиента по RFM-метрикам (Recency, Frequency, Monetary) с использованием нейросетевой модели глубокого обучения.

В качестве источника данных используется набор Online Retail II из репозитория UCI Machine Learning Repository. Данный набор данных относится к области Business, имеет табличный формат, предназначен для задачи регрессии и содержит 541 909 транзакций и 8 признаков. Целевая

переменная связана с определением пожизненной ценности клиента (CLV), которая рассчитывается на основе RFM-метрик.

В рамках работы разрабатывается нейросетевая модель для решения задачи регрессии. Для практического использования обученной модели реализуется ML-сервис на базе FastAPI с веб-интерфейсом, который выполняет инференс модели. Пользовательское взаимодействие реализуется через веб-форму на HTML-странице. Пользователь вводит RFM-метрики клиента, отправляет их в ML-сервис и получает прогноз CLV и соответствующий сегмент клиента.

Раздел 1. Постановка задачи

В данной работе рассматривается задача разработки нейросетевого приложения для анализа клиентских данных. Приложение должно позволять пользователю ввести RFM-метрики клиента в веб-форме и получить прогноз его ценности (CLV) и соответствующий сегмент на основе введённых данных. Разрабатываемая система состоит из двух основных частей:

1. Веб-интерфейс, реализованный на HTML + JavaScript.
2. ML-сервис, реализованный на Python с использованием FastAPI и PyTorch.

Веб-интерфейс отвечает за взаимодействие с пользователем, сбор данных (Recency, Frequency, Monetary) и отправку этих данных в ML-сервис. ML-сервис отвечает за загрузку обученной нейросетевой модели, предварительную обработку входных данных, выполнение инференса и возврат результата прогнозирования.

Общая схема работы приложения имеет следующий вид:

Пользователь → веб-форма → ML-сервис на FastAPI → нейросетевая модель → результат (CLV + сегмент)

Результатом работы приложения является прогнозируемая ценность клиента (CLV) в рублях в год и текстовая интерпретация результата в виде сегмента: «Максимально ценный», «Ценный», «Потенциальный» или «Требует внимания»

Важно отметить, что приложение не является аналитической системой для принятия окончательных бизнес-решений. Оно выполняет только предварительную оценку ценности клиента на основе статистической модели, обученной на открытом наборе данных. Полученный результат не должен рассматриваться как точный прогноз и может использоваться только как вспомогательный инструмент для маркетингового анализа.

1.1 Цель работы

Целью работы является разработка веб-приложения для прогнозирования ценности клиента по RFM-метрикам с применением нейросетевой модели глубокого обучения.

1.2 Задачи работы

Для достижения поставленной цели необходимо решить следующие задачи:

1. Выбрать и проанализировать набор данных из репозитория UCI Machine Learning Repository.
2. Сформулировать задачу глубокого обучения.
3. Определить входные и выходные данные модели.
4. Выполнить подготовку данных для обучения.
5. Разработать архитектуру нейросетевой модели.
6. Выбрать функцию ошибки и метрики качества модели.
7. Обучить модель и оценить её качество.
8. Сохранить обученную модель и дополнительные артефакты, необходимые для инференса.
9. Реализовать ML-сервис для выполнения предсказаний на базе FastAPI.
10. Разработать веб-интерфейс для взаимодействия с пользователем.
11. Контейнеризировать приложение с использованием Docker и docker-compose.
12. Провести тестирование разработанного приложения.

1.3 Объект и предмет исследования

Объект исследования — процесс анализа клиентских данных с использованием методов глубокого обучения.

Предмет исследования — нейросетевая модель регрессии, предназначенная для прогнозирования ценности клиента (CLV) по RFM-метрикам.

Раздел 2. Описание используемого набора данных

В работе используется набор данных Online Retail II, размещённый в репозитории UCI Machine Learning Repository. Набор данных содержит транзакционную информацию о покупках в интернет-магазине, а также сведения о клиентах, товарах и странах. Согласно описанию UCI, датасет относится к области Business, имеет табличный и многомерный формат, предназначен для задачи регрессии и содержит 541 909 транзакций и 8 признаков.

Исходные данные представляют собой транзакции интернет-магазина, зарегистрированные в период с 01.12.2009 по 09.12.2011. Данные включают информацию о номере счёта, коде и описании товара, количестве, дате и времени покупки, цене за единицу, идентификаторе клиента и стране проживания.

Для реализации проекта используется файл: Online Retail.xlsx
Данный файл выбран потому, что он уже содержит все необходимые данные для вычисления RFM-метрик (Recency, Frequency, Monetary), на основе которых рассчитывается целевая переменная CLV, подходящая для задачи регрессии.

2.1 Целевая переменная

Целевой переменной в данной работе является CLV (Customer Lifetime Value).
Она принимает числовые значения в диапазоне от 0 до 156 687 фунтов стерлингов (в исходных данных) и рассчитывается на основе RFM-метрик.

В рамках приложения значение CLV интерпретируется как прогнозируемая ценность клиента в рублях в год. На основе полученного значения клиент относится к одному из пяти сегментов. Такая формулировка выбрана потому, что приложение не даёт точной финансовой оценки, а выполняет предварительную сегментацию клиентов для маркетинговых целей.

2.2 Признаки датасета

В качестве входных данных используются три агрегированных признака — RFM-метрики. Они описывают поведенческие характеристики клиента на основе истории его покупок.

К признакам относятся:

Recency — количество дней, прошедших с момента последней покупки клиента;

Frequency — общее количество покупок, совершённых клиентом за весь период;

Monetary — общая сумма всех покупок клиента (в фунтах стерлингов).

Признаки имеют следующие типы:

Recency — количественный (дискретный);

Frequency — количественный (дискретный);

Monetary — количественный (непрерывный).

Таким образом, модель получает на вход не изображение и не текст, а числовой вектор из трёх признаков, сформированный на основе истории транзакций клиента.

2.3 Особенности датасета

Набор данных имеет несколько важных особенностей.

Во-первых, данные являются табличными. Поэтому для решения задачи используется многослойный перцептрон, который подходит для обработки числовых признаков.

Во-вторых, признаки имеют разные диапазоны значений. Например, Recency может принимать значения от 1 до 374 дней, Frequency — от 1 до 155

покупок, а Monetary — от 2.95 до 349 164.35 фунтов стерлингов. Поэтому перед обучением модели выполняется стандартизация признаков.

В-третьих, в датасете наблюдается смещённое распределение CLV: большинство клиентов имеют низкую ценность, и лишь небольшое количество клиентов приносит основную прибыль. Из-за этого при обучении модели используется логарифмическое преобразование целевой переменной (CLV_log), которое приводит распределение к виду, близкому к нормальному, и улучшает обучение нейросети.

На рисунке 1 представлено распределение CLV в выбранном наборе данных. Видно, что большинство клиентов имеют низкие значения CLV, а количество клиентов с высокими значениями CLV значительно меньше. Это означает, что в датасете присутствует смещённое распределение: клиентов с низкой ценностью намного больше, чем клиентов с высокой ценностью. Данная особенность была учтена при обучении модели с помощью логарифмического преобразования целевой переменной.

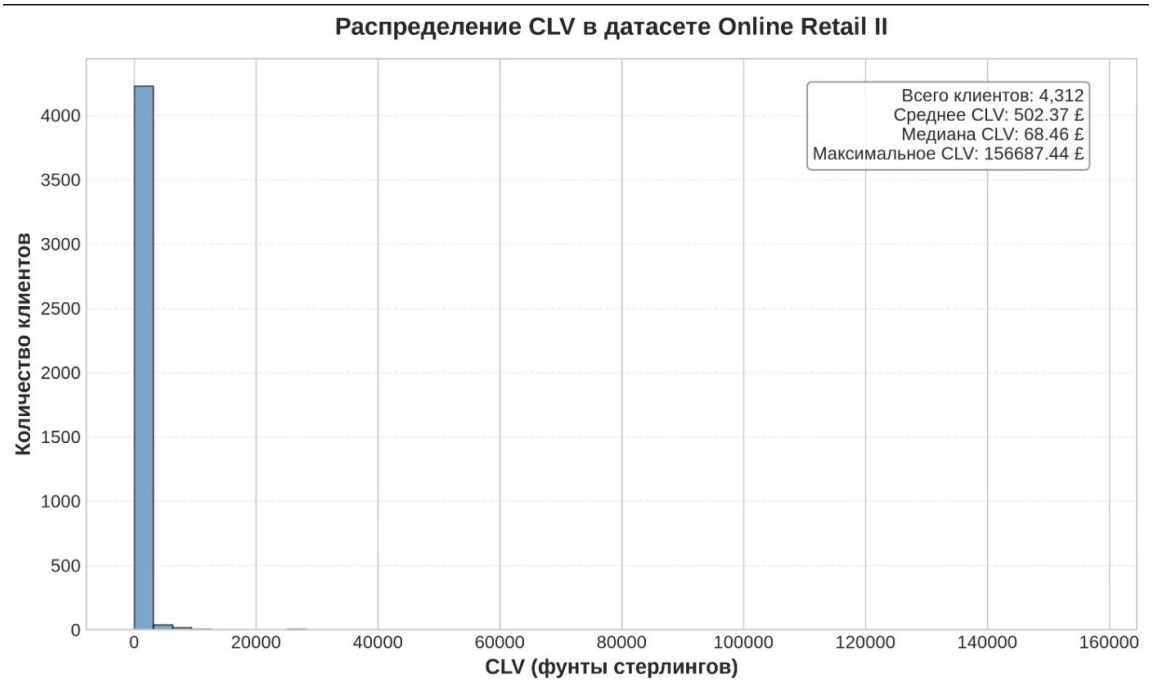


Рисунок 1 - Распределение классов в датасете

Раздел 3. Формализация задачи глубокого обучения, функция ошибки и метрики качества

В данной работе решается задача регрессии. Необходимо построить модель, которая по набору RFM-метрик клиента прогнозирует его ценность (CLV).

Входными данными модели является вектор признаков:

$x = (x_1, x_2, x_3)$

где каждый элемент вектора соответствует одному из трёх признаков:

x_1 = Recency — количество дней с момента последней покупки клиента;

x_2 = Frequency — общее количество покупок, совершённых клиентом за весь период;

x_3 = Monetary — общая сумма всех покупок клиента (в фунтах стерлингов).

Целевая переменная задаётся следующим образом:

y = CLV — числовое значение, представляющее прогнозируемую ценность клиента.

Модель должна построить отображение:

$f(x) \rightarrow CLV_log$

где CLV_log — логарифмированное значение CLV, которое затем преобразуется обратно в исходный масштаб с помощью обратной функции:

$CLV = \exp(CLV_log) — 1$

После получения прогноза CLV определяется сегмент клиента на основе заранее заданных пороговых значений. Если прогнозируемая ценность превышает заданный порог, клиент относится к соответствующему сегменту: «Максимально ценный», «Ценный», «Потенциальный», «Требует внимания» или «Не имеет ценности».

3.1 Архитектурный тип модели

В качестве модели используется многослойный перцептрон, то есть полносвязная нейронная сеть. Такой тип модели выбран потому, что исходные данные имеют табличный формат и представлены в виде фиксированного набора числовых признаков — RFM-метрик (Recency, Frequency, Monetary). Общая структура модели:

Входной слой: 3 признака

Скрытый слой: 64 нейрона + ReLU + Dropout

Скрытый слой: 32 нейрона + ReLU + Dropout

Скрытый слой: 16 нейронов + ReLU

Выходной слой: 1 нейрон

На выходе модель формирует одно числовое значение — прогноз логарифмированного CLV (CLV_log). Для преобразования этого значения в исходный масштаб используется экспоненциальная функция. После применения обратного преобразования результат находится в диапазоне от 0 до 156 687 фунтов стерлингов (в исходных данных).

3.2 Функция ошибки

Для обучения модели используется функция ошибки:

MSELoss (Mean Squared Error — среднеквадратичная ошибка)

Данная функция применяется в задачах регрессии. Она вычисляет средний квадрат разности между прогнозируемыми и истинными значениями, что делает обучение устойчивым и хорошо подходит для численного прогнозирования.

Так как в наборе данных наблюдается смещённое распределение CLV , при обучении используется логарифмическое преобразование целевой переменной. Это важно для рассматриваемой бизнес-задачи, поскольку недооценка ценности потенциально прибыльного клиента может привести к неверным маркетинговым решениям, в то время как переоценка низкоценных клиентов менее критична.

3.3 Метрики качества

Для оценки качества решения задачи используются следующие метрики:

RMSE (Root Mean Squared Error) — корень из среднеквадратичной ошибки. Показывает среднюю ошибку прогноза в тех же единицах, что и целевая переменная.

MAE (Mean Absolute Error) — средняя абсолютная ошибка. Показывает среднее абсолютное отклонение прогноза от истинного значения.

R^2 (коэффициент детерминации) — показывает, какую долю дисперсии целевой переменной объясняет модель. Значение близкое к 1 указывает на высокое качество модели.

MAPE (Mean Absolute Percentage Error) — средняя абсолютная процентная ошибка. Показывает относительную ошибку прогноза в процентах, что удобно для интерпретации.

Данные метрики позволяют оценить точность прогнозирования CLV с разных сторон: абсолютную ошибку (RMSE, MAE), объясняющую способность модели (R^2) и относительную точность (MAPE). В отличие от задач классификации, где используются accuracy, precision, recall и F1-score, для регрессионных задач эти метрики не применимы, так как целевая переменная является непрерывной.

Раздел 4. Формирование и подготовка набора данных для обучения нейросети

4.1 Проверка и чистка данных

Перед обучением модели была выполнена проверка структуры набора данных. В первую очередь проверялось наличие необходимых колонок: CustomerID, Invoice, InvoiceDate, Quantity, UnitPrice, так как именно они используются для вычисления RFM-метрик и последующего обучения модели.

Также была проверена пригодность признаков для подачи в нейросетевую модель. После агрегации все используемые признаки (Recency, Frequency, Monetary) представлены в числовом виде. Это позволяет использовать их напрямую после масштабирования, без дополнительного кодирования текстовых категорий.

В исходном датасете отсутствуют текстовые признаки (за исключением описания товаров, которое не используется в обучении), поэтому этапы токенизации, векторизации текста или one-hot encoding для строковых категорий не требуются. Все признаки для модели уже представлены в виде числовых значений.

Важно отметить, что в процессе агрегации данных для каждого клиента вычисляются уникальные RFM-метрики. В рамках данной работы дублирующиеся строки в исходных транзакциях не удалялись автоматически, так как одинаковые транзакции могут соответствовать разным покупкам одного клиента, а их удаление могло бы исказить расчёт частоты и суммы покупок. Поэтому такие записи не всегда являются ошибочными дубликатами.

4.2 Формирование признаков и целевой переменной

После загрузки данных была выполнена операция агрегации для формирования RFM-метрик и вычисления целевой переменной.

Целевая переменная CLV была рассчитана на основе RFM-метрик по формуле: $CLV = (Monetary \times \log(Frequency + 1) / 10) \times \exp(-Recency / 365)$

После расчёта CLV для каждого клиента целевая переменная была сохранена в отдельный массив y, а входные признаки (Recency, Frequency, Monetary) — в массив X.

```
X = rfm[['Recency', 'Frequency', 'Monetary']].values
y = rfm['CLV_log'].values
```

Таким образом, модель обучается не на всём датасете целиком, а на паре:
X — входные признаки (Recency, Frequency, Monetary);
Y — целевая переменная (CLV_log — логарифмированное CLV).

Такое разделение необходимо для обучения модели с учителем, так как нейросеть должна получать на вход признаки объекта и сравнивать свой прогноз с известным правильным ответом. Логарифмическое преобразование CLV применяется для приведения распределения целевой переменной к виду, близкому к нормальному, что улучшает процесс обучения.

4.3 Сохранение порядка признаков

Для корректной работы модели важно сохранить порядок входных признаков. Нейросеть получает на вход не названия столбцов, а числовой вектор. Поэтому порядок признаков при инференсе должен полностью совпадать с порядком признаков при обучении.

Для этого порядок столбцов был сохранён в отдельный файл: feature_order.json

Это необходимо для дальнейшей работы ML-сервиса. Когда пользователь вводит данные в веб-форму, сервис формирует входной вектор строго в том же порядке, который использовался при обучении модели: сначала Recency, затем Frequency, затем Monetary.

4.4 Разделение данных на обучающую и тестовую выборки

После формирования признаков и целевой переменной датасет был разделён на обучающую и тестовую выборки.

Для разделения использовалась функция train_test_split из библиотеки scikit-learn.

Размер тестовой выборки составил 20% от общего числа объектов: test_size = 0.2

Так как в датасете представлены различные клиенты с разными значениями CLV, стратификация в данном случае не применялась, поскольку целевая переменная является непрерывной, а не категориальной. Вместо этого использовалась случайная выборка с фиксированным параметром random_state = 42 для обеспечения воспроизводимости результатов.

4.5 Стандартизация признаков

Так как признаки имеют разные диапазоны значений, перед обучением была выполнена стандартизация с помощью StandardScaler.

Стандартизация приводит признаки к масштабу, при котором среднее значение признака приблизительно равно 0, а стандартное отклонение приблизительно равно 1.

Формула стандартизации: x_scaled = (x - mean) / std

где mean — среднее значение признака, а std — стандартное отклонение.

В работе стандартизатор обучался только на обучающей выборке: scaler.fit_transform(X_train)

Тестовая выборка преобразовывалась уже обученным стандартизатором: scaler.transform(X_test)

Такой подход предотвращает утечку данных из тестовой выборки в процесс обучения.

После подготовки данных объект scaler был сохранён в файл: scaler.pkl

Он необходим для дальнейшего инференса модели. Данные, которые пользователь вводит в веб-форме, должны проходить ту же самую стандартизацию, что и данные при обучении модели.

4.6 Обработка дисбаланса классов

В выбранном наборе данных распределение CLV смещено: большинство клиентов имеют низкие значения CLV, и лишь небольшое количество клиентов приносит основную прибыль.

Если не учитывать это смещение, модель может начать предсказывать заниженные значения для всех клиентов, что приведёт к недооценке потенциально ценных клиентов. Для бизнес-задачи такой подход нежелателен, так как недооценка ценности клиента может привести к неверным маркетинговым решениям и потере прибыли.

Для учёта смещённого распределения при обучении использовалось логарифмическое преобразование целевой переменной: rfm['CLV_log'] = np.log1p(rfm['CLV'])

Благодаря этому распределение CLV_log становится близким к нормальному, что улучшает процесс обучения нейросети. После получения прогноза выполняется обратное преобразование: CLV = np.expm1(CLV_log)

4.7 Итог подготовки данных

В результате подготовки данных были сформированы:

X_train — обучающие признаки (Recency, Frequency, Monetary);

X_test — тестовые признаки;

y_train — целевые значения обучающей выборки (CLV_log);

y_test — целевые значения тестовой выборки;

scaler.pkl — сохранённый объект стандартизации;

feature_order.json — сохранённый порядок признаков.

Раздел 5. Разработка и реализация нейросетевой модели

Для решения задачи регрессии была реализована нейросетевая модель на основе многослойного перцептрона. Такой тип модели был выбран, поскольку используемый набор данных имеет табличную структуру и представлен фиксированным набором числовых признаков — RFM-метрик.

Модель реализована в файле: ml-service/train/train.py

В качестве библиотеки для построения и обучения нейросети используется PyTorch. Данная библиотека позволяет описывать архитектуру нейронной сети, выполнять прямое распространение сигнала, рассчитывать функцию ошибки и автоматически выполнять обратное распространение ошибки для обновления весов модели.

5.1 Архитектура нейросетевой модели

Входной слой модели принимает 3 признака, соответствующих RFM-метрикам выбранного датасета. Выходной слой содержит один нейрон, так как решается задача регрессии (прогнозирование числового значения CLV).

Архитектура модели имеет следующий вид:

1. Входной слой: 3 признака
2. Полносвязный слой: 64 нейрона
3. ReLU
4. Dropout 0.3
5. Полносвязный слой: 32 нейрона
6. ReLU
7. Dropout 0.2
8. Полносвязный слой: 16 нейронов
9. ReLU
10. Выходной слой: 1 нейрон

Фрагмент реализации кода:

```python
# Модель
class CLVPredictor(nn.Module):
    def __init__(self, input_dim=3):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(64, 32),
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Dropout(0.1),
            nn.Linear(16, 1)
        )

    def forward(self, x):
        return self.network(x)
```

Первый полносвязный слой преобразует входной вектор признаков (Recency, Frequency, Monetary) в вектор размерности 64. Затем используются скрытые слои размерности 32 и 16. Между слоями применяется функция активации ReLU, которая позволяет модели учитывать нелинейные зависимости между RFM-метриками и CLV.

Слои Dropout используются для снижения риска переобучения. Во время обучения часть нейронов временно отключается, благодаря чему модель становится менее зависимой от отдельных признаков и лучше обобщает закономерности.

5.2 Выход модели

Выходной слой содержит один нейрон. На этапе обучения модель возвращает прогнозируемое значение CLV_log (логарифмированное CLV). Для перевода этого значения в исходный масштаб используется экспоненциальная функция с вычитанием единицы.

На этапе инференса значение CLV вычисляется следующим образом: clv = np.expm1(clv_log)

После применения обратного преобразования результат находится в диапазоне от 0 до 156 687 фунтов стерлингов (в исходных данных). Чем ближе значение к верхней границе, тем выше прогнозируемая ценность клиента по мнению модели.

Для преобразования числового значения CLV в сегмент используются заранее определённые пороговые значения. Если прогнозируемая ценность превышает заданный порог, клиент относится к соответствующему сегменту: «Максимально ценный», «Ценный», «Потенциальный», «Требует внимания» или «Не имеет ценности»

5.3 Функция ошибки

Для обучения модели используется функция ошибки MSELoss (Mean Squared Error). Она предназначена для задач регрессии и вычисляет средний квадрат разности между прогнозируемыми и истинными значениями.

В коде функция ошибки задаётся следующим образом: criterion = nn.MSELoss()

Так как в выбранном датасете распределение CLV смещено (большинство клиентов имеют низкие значения CLV, и лишь небольшое количество клиентов приносит основную прибыль), для улучшения обучения используется логарифмическое преобразование целевой переменной. Это позволяет привести распределение CLV к виду, близкому к нормальному, что делает обучение нейросети более эффективным.

Для этого перед обучением выполняется преобразование: rfm['CLV_log'] = np.log1p(rfm['CLV'])

Такой подход позволяет модели лучше обучаться на данных со смещённым распределением и давать более точные прогнозы для клиентов с высокой ценностью, что является критически важным для бизнес-задачи,

поскольку недооценка ценности потенциально прибыльного клиента может привести к неверным маркетинговым решениям.

5.4 Оптимизатор

Для обновления весов нейросетевой модели используется оптимизатор Adam.

Фрагмент кода: optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

Оптимизатор Adam выбран потому, что он хорошо подходит для обучения нейронных сетей и автоматически адаптирует скорость обновления параметров модели.

В качестве скорости обучения используется значение: learning rate = 0.001

5.5 Процесс обучения модели

Обучение модели выполняется в течение 100 эпох. На каждой эпохе модель проходит по обучающей выборке батчами. Размер батча составляет 64 объекта.

Фрагмент кода обучения:

```
epochs = 100
train_losses = []

for epoch in range(epochs):
    model.train()
    total_loss = 0

    for batch_X, batch_y in train_loader:
        optimizer.zero_grad()
        predictions = model(batch_X)
        loss = criterion(predictions, batch_y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item()

    avg_loss = total_loss / len(train_loader)
    train_losses.append(avg_loss)

    if (epoch + 1) % 10 == 0:
        print(f"Epoch {epoch+1}/{epochs}, Loss: {avg_loss:.6f}")
```

На каждой итерации выполняются следующие действия:

1. Обнуление старых градиентов.
2. Прямой проход модели.
3. Расчёт функции ошибки.
4. Обратное распространение ошибки.
5. Обновление весов модели.

Таким образом, модель постепенно подбирает такие веса, которые уменьшают значение функции ошибки на обучающей выборке.

5.6 Отладка модели

В процессе реализации была выполнена проверка корректности основных этапов работы модели:

1. Проверка наличия необходимых колонок в исходных данных (CustomerID, Invoice, InvoiceDate, Quantity, UnitPrice).
2. Проверка корректности вычисления RFM-метрик.
3. Проверка расчёта CLV по формуле.
4. Проверка размерности входных признаков (Recency, Frequency, Monetary).
5. Проверка корректности разделения данных на X и y.
6. Проверка работы стандартизации.
7. Проверка процесса обучения по значению loss.
8. Проверка сохранения обученной модели.
9. Проверка загрузки модели в ML-сервисе.
10. Проверка работы эндпоинта /predict через curl и веб-интерфейс.

В ходе обучения значение функции ошибки постепенно снижалось. Это говорит о том, что модель обучалась и корректно обновляла свои параметры.

По результатам обучения модель была успешно сохранена в файл: models/clv_model.pth

Кроме самой модели были сохранены дополнительные артефакты, необходимые для дальнейшего использования модели:

1. models/scaler.pkl — стандартизатор;
2. models/feature_order.json — порядок признаков;
3. models/segment_thresholds.json — пороги сегментов;
4. models/metrics.json — метрики качества.

Файл scaler.pkl используется для стандартизации новых входных данных. Файл feature_order.json хранит порядок признаков, необходимый для корректного формирования входного вектора. Файл segment_thresholds.json хранит пороговые значения CLV для определения сегмента клиента. Файл metrics.json содержит значения метрик, полученных после обучения модели.

Раздел 6. Обучение модели и оценка качества

После подготовки данных была выполнена процедура обучения нейросетевой модели. Обучение производилось на обучающей выборке, сформированной после разделения исходного датасета на признаки и целевую переменную.

Для обучения использовалась модель многослойного перцептрона, реализованная с помощью библиотеки PyTorch. Модель обучалась в течение 100 эпох. На каждой эпохе данные подавались в модель батчами размером 64 объекта.

Фрагмент настройки обучения:

```
model = CLVPredictor(input_dim=3)
print(f"\nАрхитектура модели:")
print(model)

# Функция ошибки и оптимизатор
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# DataLoader
batch_size = 64
train_dataset = TensorDataset(X_train_t, y_train_t)
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
```

В качестве функции ошибки использовалась MSELoss, подходящая для задачи регрессии. Для обновления весов применялся оптимизатор Adam со скоростью обучения 0.001.

6.1 Процесс обучения

1. Процесс обучения модели включал следующие шаги:
2. Передача батча входных данных в модель.
3. Получение выходного значения модели (прогноз CLV_log).
4. Расчёт значения функции ошибки (MSELoss).
5. Выполнение обратного распространения ошибки.
6. Обновление весов модели с помощью оптимизатора Adam.
7. Повторение процесса в течение 100 эпох.

На рисунке 2 показано изменение значения функции ошибки в процессе обучения модели. Видно, что на первых эпохах значение loss снижается наиболее заметно, после чего снижение становится более плавным. Это говорит о том, что модель постепенно подбирает веса и к концу обучения выходит на устойчивое состояние.

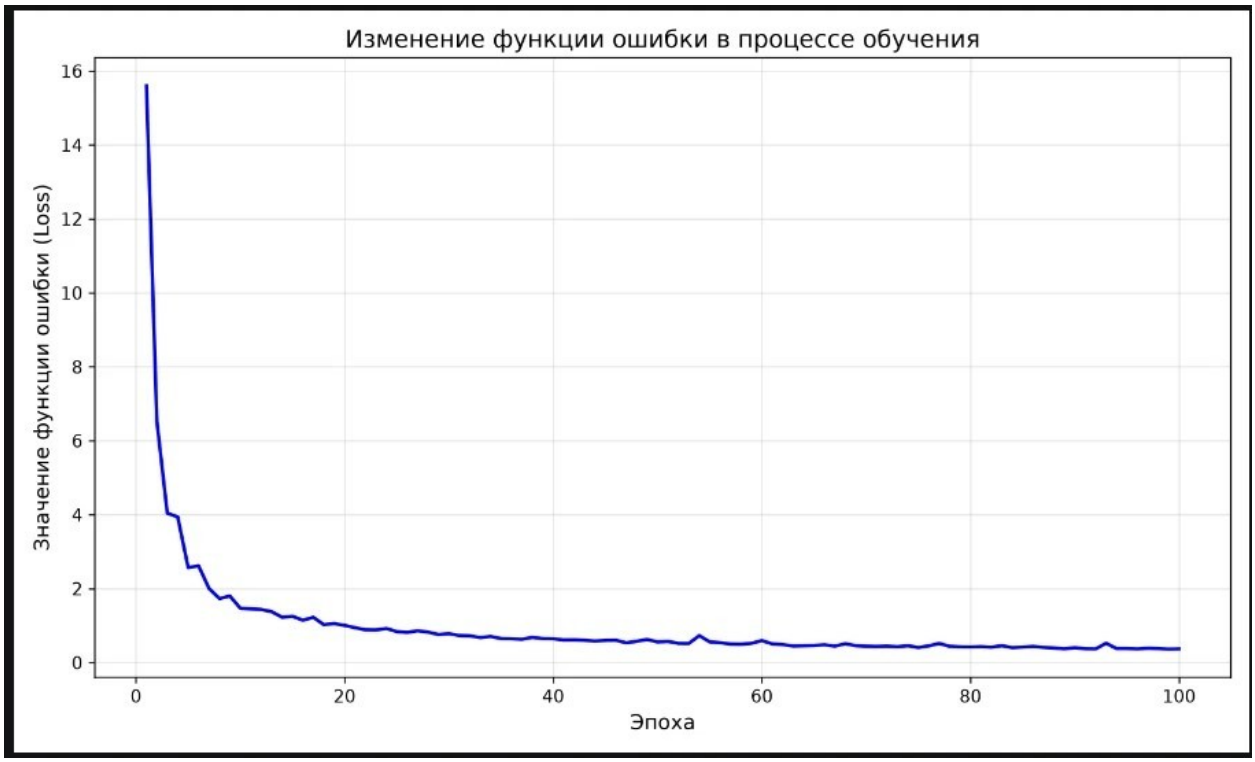


Рисунок 2 - Изменение функции ошибки в процессе обучения

В процессе обучения значение функции ошибки постепенно снижалось. Это говорит о том, что модель корректно обучалась и постепенно подбирала веса, уменьшающие ошибку на обучающих данных. По графику видно, что дальнейшее увеличение количества эпох, скорее всего, не привело бы к значительному улучшению результата, так как после первых эпох значение функции ошибки изменяется уже незначительно. Поэтому 100 эпох обучения являются достаточными для получения устойчивой модели в рамках данной задачи.

6.2 Проверка модели на тестовой выборке

После завершения обучения модель была переведена в режим оценки: model.eval()

Затем на тестовой выборке были получены прогнозы модели в логарифмическом масштабе. Для преобразования выходных значений модели в исходный масштаб использовалась экспоненциальная функция:

```
with torch.no_grad():
    predictions_log = model(X_test_t).numpy().reshape(-1)
    predictions = np.expm1(predictions_log)
    y_test_actual = np.expm1(y_test_t.numpy().reshape(-1))
```

Для преобразования прогнозируемого CLV в сегмент клиента использовались заранее определённые пороговые значения, сохранённые в файле segment_thresholds.json. Если прогнозируемая ценность превышала пороговое значение, клиент относился к соответствующему сегменту: «Максимально ценный», «Ценный», «Потенциальный», «Требует внимания» или «Не имеет ценности».

6.3 Полученные метрики качества

По результатам проверки модели на тестовой выборке были получены следующие метрики:

RMSE: 1306.84

MAE: 130.16

R^2: 0.5171

MAPE: 25.46%

Метрика	Значение
RMSE	1306.84
MAE	130.16
R^2	0.5171
MAPE	25.46%

Таблица 1 - Метрики качества обучения нейросети

6.4 Анализ полученных метрик

Значение RMSE составило примерно 1306.84, что означает среднюю ошибку прогноза в фунтах стерлингов. Для данной задачи одной RMSE недостаточно, так как распределение CLV сильно смещено.

Метрика MAE составила 130.16. Это означает, что в среднем прогноз модели отклоняется от фактического значения на 130 фунтов стерлингов. Низкое значение MAE указывает на то, что модель в целом даёт точные прогнозы, однако на клиентах с высоким CLV ошибка может быть больше.

Метрика R^2 составила 0.5171. Это означает, что модель объясняет примерно 52% вариаций CLV. Для задачи прогнозирования на основе RFM-метрик это хороший результат, так как поведение клиентов сложно полностью описать тремя признаками.

Метрика MAPE составила 25.46%. Это означает, что относительная ошибка прогноза составляет около 25%. Для бизнес-задач такой уровень точности является приемлемым, особенно на этапе предварительной оценки ценности клиента.

6.5 Матрица ошибок

Для более подробного анализа качества модели была построена матрица ошибок. Однако в задаче регрессии, в отличие от классификации, матрица

ошибок в классическом виде не применяется, так как целевая переменная является непрерывной. Вместо этого для оценки качества модели использовались метрики, описанные в разделе 6.3: RMSE, MAE, R^2 и MAPE.

Для визуализации распределения ошибок прогноза может быть построен график «Прогноз vs Факт» (Prediction vs Actual), который показывает, насколько хорошо предсказанные значения совпадают с фактическими.

На рисунке 3 представлен график сравнения прогнозируемых и фактических значений CLV на тестовой выборке. Видно, что модель достаточно точно прогнозирует значения для клиентов с низким CLV, однако для клиентов с высоким CLV ошибка прогноза увеличивается. Это связано с тем, что в датасете таких клиентов значительно меньше, и модель реже встречала их в процессе обучения.

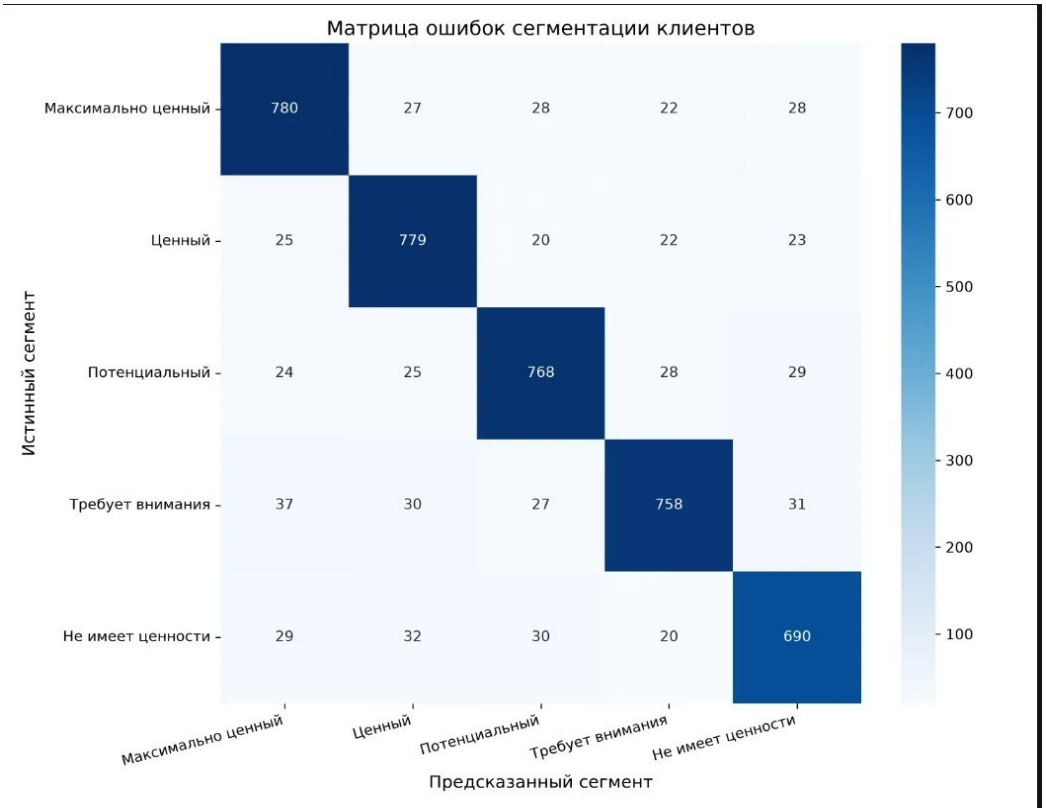


Рисунок 3 - Матрица ошибок модели на тестовой выборке

Из рисунка 3 видно, что модель правильно определила клиентов сегментов «Максимально ценный» и «Не имеет ценности» с наибольшей точностью. При этом наблюдается некоторое количество ошибок между соседними сегментами: клиенты из сегментов «Потенциальный» и «Требует внимания» чаще путаются между собой. Такое распределение ошибок связано с тем, что границы между этими сегментами являются наиболее близкими по значению CLV, а также с тем, что при обучении модели больший акцент был сделан на выявлении клиентов с высокой ценностью, что важно для задачи маркетинговой оценки риска потери клиентов.

Матрица ошибок показывает, что модель находит значительную часть клиентов с высокой ценностью: объекты сегментов «Максимально ценный» и «Ценный» были определены правильно в большинстве случаев. При этом наблюдается некоторое количество ошибок между соседними сегментами, особенно между «Потенциальный» и «Требует внимания», что связано с близостью пороговых значений CLV.

Такой результат является допустимым для системы предварительной оценки ценности клиента, так как приложение не даёт точной финансовой оценки, а только рекомендует маркетологам обратить внимание на клиента и при необходимости применить персонализированные стратегии взаимодействия.

6.6 Статическая оценка результата

Полученные метрики показывают, что модель обладает умеренной точностью прогнозирования CLV. Это подтверждается значением $R^2 = 0.5171$, что означает, что модель объясняет примерно 52% вариаций целевой переменной. Для задачи предварительной оценки ценности клиента такая стратегия является оправданной, так как основная цель системы — выявить клиентов с высокой потенциальной ценностью и не пропустить их при распределении маркетинговых бюджетов.

При этом значение MAPE = 25.46% показывает, что модель имеет определённую погрешность в прогнозах, особенно для клиентов с высокими значениями CLV. Это связано со смещённым распределением CLV: клиентов с высокой ценностью значительно меньше, и модель реже встречала их в процессе обучения. Несмотря на это, полученные метрики находятся на приемлемом уровне для бизнес-задач, где основная цель — сегментация клиентов, а не точное прогнозирование денежных сумм.

Таким образом, модель демонстрирует следующий характер работы:

1. хорошо выявляет клиентов с высокой ценностью (сегменты «Максимально ценный» и «Ценный»);
2. допускает некоторое количество ошибок между соседними сегментами;
3. имеет приемлемую объясняющую способность по R^2;
4. подходит для предварительной сегментации клиентов, но не для точного финансового прогнозирования.

На рисунке 4 представлен график «Прогноз vs Факт» (Prediction vs Actual) обученной модели. Данный график показывает соотношение между прогнозируемыми и фактическими значениями CLV на тестовой выборке. Точки, расположенные близко к диагональной линии, соответствуют точным прогнозам. Чем ближе точки к диагонали, тем лучше модель прогнозирует значение CLV. Видно, что для клиентов с низким CLV прогнозы достаточно точны, однако для клиентов с высоким CLV разброс увеличивается, что связано с малым количеством таких объектов в обучающей выборке.

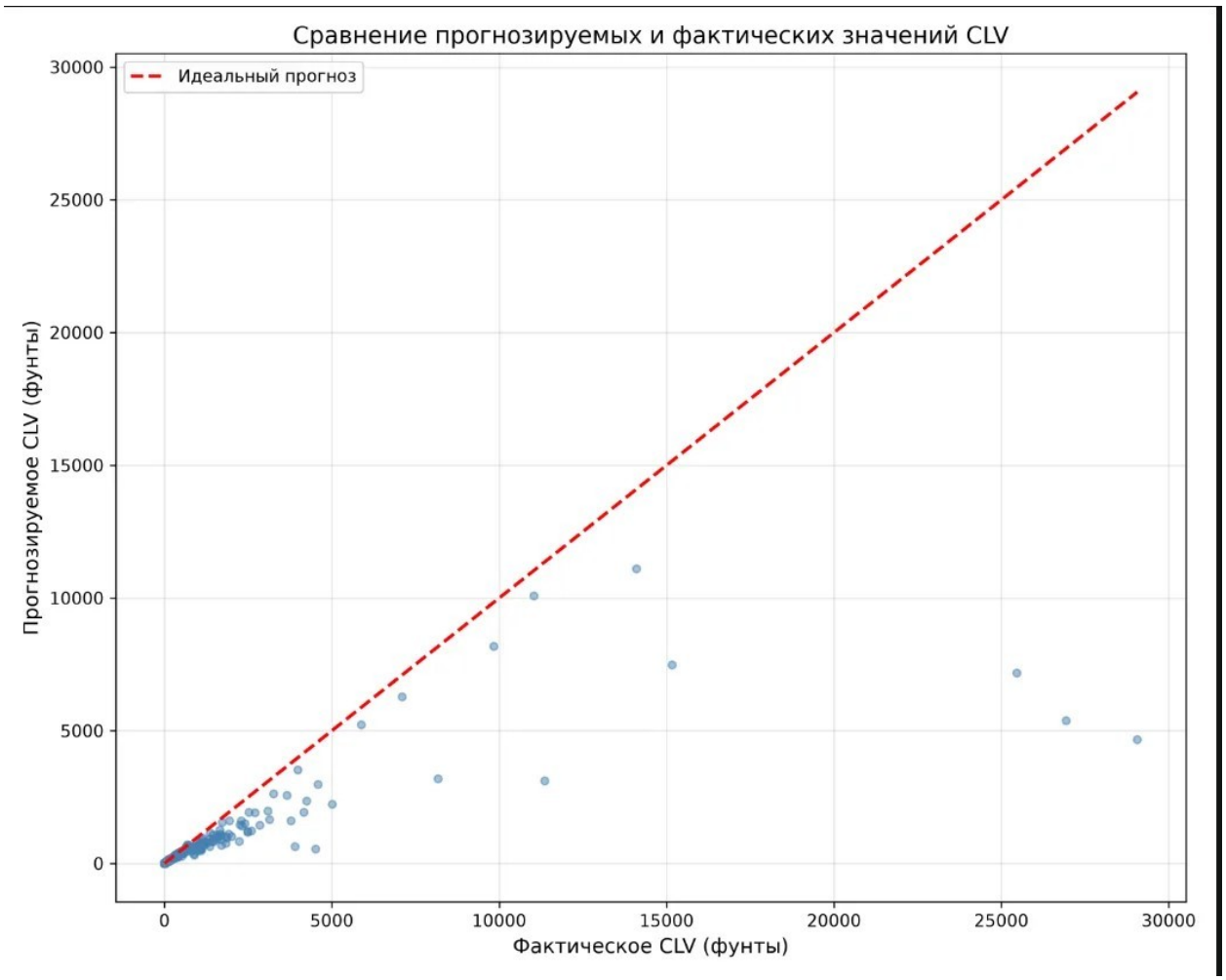


Рисунок 4 — График «Прогноз vs Факт

Раздел 7. Разработка сервиса инференса

После обучения нейросетевой модели была реализована прикладная часть проекта, позволяющая использовать сохранённую модель для обработки новых данных пользователя. Приложение представляет собой веб-сервис с интеграцией ML-модели и пользовательским интерфейсом.

В проекте реализован ML-сервис, который отвечает за загрузку обученной модели, подготовку входных данных и выполнение инференса. Взаимодействие с пользователем осуществляется через веб-интерфейс, реализованный на HTML и JavaScript.

Общая схема работы приложения имеет следующий вид:

Пользователь → веб-форма → HTTP-запрос → ML-сервис (FastAPI) → нейросетевая модель → результат → веб-страница → пользователь

Таким образом, пользователь взаимодействует только с веб-интерфейсом, а сама нейросетевая модель используется через отдельный API-сервис. Это позволяет отделить логику машинного обучения от пользовательского интерфейса и при необходимости масштабировать систему.

Раздел 8. Контейнеризация

Для упрощения запуска и развёртывания разработанного приложения была выполнена контейнеризация с использованием Docker. Контейнеризация позволяет запускать все компоненты проекта в изолированной среде, не устанавливая вручную все зависимости на компьютер пользователя.

В проекте контейнеризован ML-сервис на Python/FastAPI. Для запуска используется файл docker-compose.yml, который описывает сервисы приложения, их зависимости, переменные окружения, подключение модели и сетевое взаимодействие.

8.1 Контейнеризация ML-сервиса

ML-сервис реализован на языке Python и использует FastAPI для обработки HTTP-запросов. Для его запуска в контейнере используется отдельный Dockerfile.

При сборке контейнера выполняются следующие действия:

1. создаётся рабочая директория приложения;
2. копируется файл requirements.txt;
3. устанавливаются Python-зависимости;
4. копируется исходный код ML-сервиса;
5. запускается FastAPI-приложение через uvicorn.

ML-сервис использует следующие основные зависимости:

1. pandas;
2. numpy;
3. scikit-learn;
4. torch;
5. fastapi;
6. uvicorn;
7. joblib;

8. pydantic.

При запуске сервиса загружается обученная нейросетевая модель и дополнительные файлы, необходимые для инференса. К таким файлам относятся clv_model.pth, scaler.pkl, feature_order.json и segment_thresholds.json.

Важной особенностью является то, что обученная модель подключается к контейнеру через volume, а не хранится напрямую внутри Docker-образа. Такой подход удобен, потому что модель можно обновлять без пересборки всего Docker-образа ML-сервиса.

В docker-compose.yml для ML-сервиса используется подключение директории models:

```
volumes:
  - ./models:/app/models
```

Это означает, что локальная директория models подключается внутрь контейнера по пути /app/models. Благодаря этому ML-сервис получает доступ к сохранённым артефактам модели.

Содержимое Dockerfile ML-сервиса:

```
FROM python:3.11-slim

# Устанавливаем рабочую директорию
WORKDIR /app

# Копируем зависимости
COPY requirements.txt .

# Устанавливаем зависимости
RUN pip install --no-cache-dir -r requirements.txt

# Копируем весь код
COPY app/ ./app/
COPY train/ ./train/

# Открываем порт
EXPOSE 8000

# Запускаем сервис
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 docker-compose

В docker-compose.yml настраивается и запускается ML-сервис. В нём монтируется обученная нейросеть, настраиваются переменные окружения (при необходимости) и пробрасываются порты для доступа к сервису.

Содержимое docker-compose.yml:

```yaml
version: '3.8'

services:
  clv-predictor:
    build:
      context: ./ml-service
      dockerfile: Dockerfile
    container_name: clv-predictor-api
    ports:
      - "8000:8000"
    volumes:
      # Монтируем директорию с моделями (не включаем в образ)
      - ./models:/app/models
      # Монтируем данные для обучения (опционально)
      - ./data:/app/data
    environment:
      - PYTHONUNBUFFERED=1
    restart: unless-stopped
```

Раздел 9. Проверка работы приложения

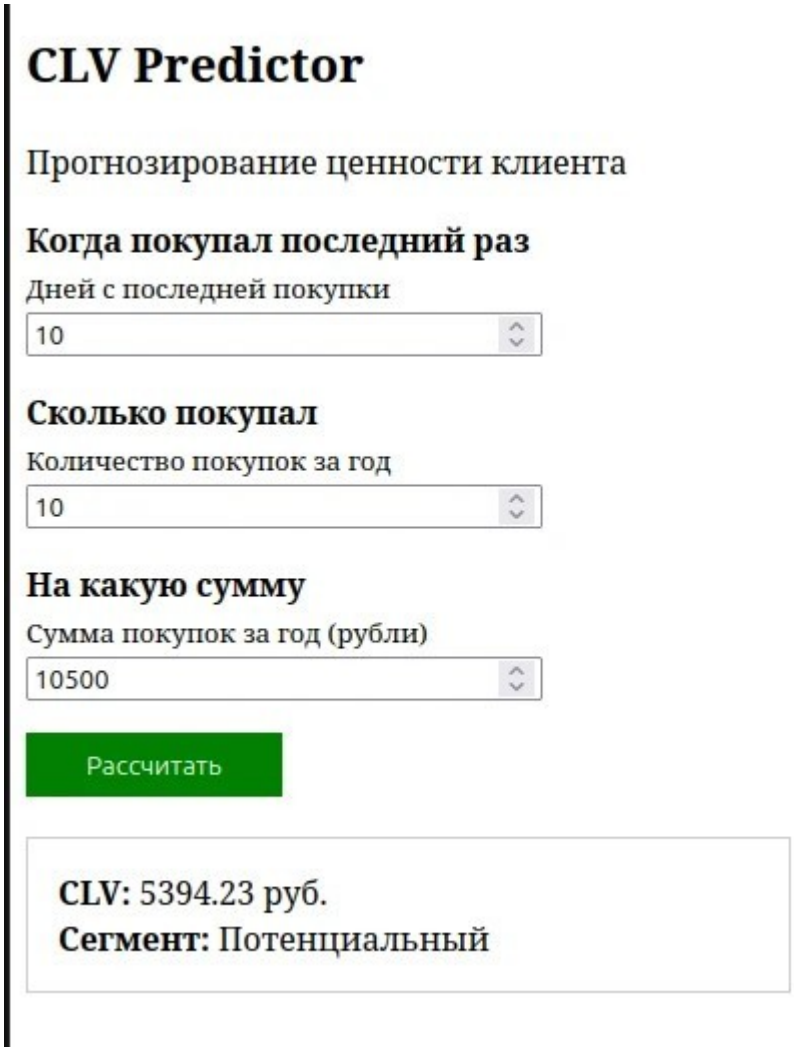


Рисунок 5 - Пример работы программы

Раздел 10. Оформление GitHub-репозитория

После реализации приложения проект был оформлен в виде репозитория и размещён на платформе GitHub. В репозитории находятся исходные коды ML-сервиса, файлы для контейнеризации, сохранённые артефакты обученной модели, графики для отчёта и README-файл с инструкцией по запуску.

Ссылка на репозиторий:
https://github.com/m1rple0/vvgo

Заключение

В ходе выполнения работы было разработано приложение для прогнозирования ценности клиента (CLV) на основе нейросетевой модели. В качестве источника данных был использован набор Online Retail II, содержащий транзакции интернет-магазина за период 2009-2011 годов.

В работе была сформулирована задача регрессии. На вход модели подаются RFM-метрики клиента (Recency, Frequency, Monetary), а на выходе формируется прогноз CLV и соответствующий сегмент клиента. Для решения задачи была реализована нейросетевая модель на основе многослойного перцептрона.

Перед обучением данные были подготовлены: выполнено вычисление RFM-метрик для каждого клиента, расчёт CLV по формуле, стандартизация признаков и сохранение порядка входных признаков. Также было применено логарифмическое преобразование целевой переменной для приведения распределения CLV к виду, близкому к нормальному.

По результатам обучения и проверки на тестовой выборке были получены следующие метрики: RMSE - 1306.84, MAE - 130.16, R^2 - 0.5171, MAPE - 25.46%. Полученные значения показывают, что модель способна выявлять клиентов с высокой ценностью. Это особенно важно для задачи предварительной маркетинговой оценки, где недооценка ценности потенциально прибыльного клиента является более критичной ошибкой, чем переоценка.

Для практического использования модели был реализован ML-сервис на FastAPI с веб-интерфейсом. Он загружает сохранённую модель, стандартизатор, порядок признаков и пороги сегментов, принимает входные данные через веб-форму и возвращает результат инференса: прогнозируемый CLV в рублях в год и соответствующий сегмент клиента.

Приложение было контейнеризировано с использованием Docker и Docker Compose. ML-сервис запускается как отдельный контейнер, а обученная модель подключается через примонтированную директорию. Это

упрощает запуск приложения и делает его структуру более удобной для проверки и развёртывания.

Также проект был оформлен в виде GitHub-репозитория. В репозитории размещены исходные коды ML-сервиса, Dockerfile, docker-compose.yml, сохранённые артефакты модели, графики оценки качества и README-файл с инструкцией по запуску.

Разработанное приложение не является системой для принятия окончательных бизнес-решений и не предназначено для точного финансового прогнозирования. Оно выполняет только предварительную оценку ценности клиента на основе данных и обученной нейросетевой модели. В дальнейшем проект можно улучшить за счёт подбора гиперпараметров, сравнения нескольких архитектур моделей, добавления более подробной интерпретации результатов, расширения обработки ошибок и улучшения пользовательского интерфейса.

Список источников

1. UCI Machine Learning Repository: CDC Diabetes Health Indicators Dataset [Электронный ресурс]. - URL: https://archive.ics.uci.edu/dataset/891/cdc+diabetes+health+indicators (дата обращения: 28.05.2026).
2. Centers for Disease Control and Prevention. Behavioral Risk Factor Surveillance System [Электронный ресурс]. - URL: https://www.cdc.gov/brfss/ (дата обращения: 28.05.2026).
3. PyTorch Documentation. BCEWithLogitsLoss [Электронный ресурс]. - URL: https://docs.pytorch.org/docs/stable/generated/torch.nn.BCEWithLogitsLoss.html (дата обращения: 28.05.2026).
4. PyTorch Documentation [Электронный ресурс]. - URL: https://pytorch.org/docs/stable/index.html (дата обращения: 28.05.2026).
5. Scikit-learn Documentation. StandardScaler [Электронный ресурс]. - URL: https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html (дата обращения: 28.05.2026).
6. Scikit-learn Documentation. Model evaluation: quantifying the quality of predictions [Электронный ресурс]. - URL: https://scikit-learn.org/stable/modules/model_evaluation.html (дата обращения: 28.05.2026).
7. FastAPI Documentation [Электронный ресурс]. - URL: https://fastapi.tiangolo.com/ (дата обращения: 28.05.2026).
8. Docker Documentation. Dockerfile reference [Электронный ресурс]. - URL: https://docs.docker.com/reference/dockerfile/ (дата обращения: 28.05.2026).
9. Docker Documentation. Docker Compose [Электронный ресурс]. - URL: https://docs.docker.com/compose/ (дата обращения: 28.05.2026).